



STROJNO UČENJE
13. predavanje
 Modeli koji za nas upravljaju značajkama (SVM, PCA)

AI Center Lipik
 THE FUTURE IS HERE

13.1. Selekcija značajki

- Wine dataset (target: 0,1,2)

Broj instanci:	178
Broj atributa:	13 numeričkih, prediktivnih atributa i klasa
Informacije o atributima:	- Alkohol - Alkalna kiselina - Peper - Alkalnost pepela - Magnecij - Ukupni fenoli - Flavonoidi - Refrakcijski indeks - Proizvodnja - Intenzitet boje - Najam - OD280/OD315 razmjerenih vina - Proizn



Koje značajke više doprinose točnosti modela ?

- f_...** - pokriva samo linearnu vezu. F-statistika
- mutual_info_...** (međusobna informacija) – pokriva i nelinearnu vezu
 - međusobna informacija (engl. mutual information) je mjera količine informacija koju dvije slučajne varijable dijele. Ona pokazuje koliko saznanje o vrijednosti jedne varijable smanjuje nesigurnost (entropiju) o drugoj varijabli.

```
from sklearn import feature_selection as ftr_sel
import sklearn.datasets as datasets
import sklearn.model_selection as skms
import numpy as np

kwargs = dict(test_size=0.3, random_state=42)
wine = datasets.load_wine()
tts = skms.train_test_split(wine.data, wine.target, **kwargs)
(wine_train, wine_test,
 wine_train_tgt, wine_test_tgt) = tts

diabetes = datasets.load_diabetes()
tts = skms.train_test_split(diabetes.data, diabetes.target, **kwargs)
(diabetes_train_ftrs, diabetes_test_ftrs,
 diabetes_train_tgt, diabetes_test_tgt) = tts
```

Linearna veza, klasifikacija– **f_classif**

```
ftrsel = ftr_sel.SelectKBest(ftr_sel.f_classif, k=5)
ftrsel.fit_transform(wine_train, wine_train_tgt)

keepers_idx = ftrsel.get_support()
print(np.array(wine.feature_names)[keepers_idx])
```

Linearna veza, regresija– **f_regression**

```
ftrsel = ftr_sel.SelectPercentile(ftr_sel.f_regression,
                                percentile=25)
ftrsel.fit_transform(diabetes_train_ftrs,
                    diabetes_train_tgt)

print(np.array(diabetes.feature_names)[ftrsel.get_support()])
```

Nelinearne veze. Klasifikacija – **mutual_info_classif**

```
ftrsel = ftr_sel.SelectKBest(ftr_sel.mutual_info_classif, k=5)
ftrsel.fit_transform(wine_train, wine_train_tgt)

keepers_idx = ftrsel.get_support()
print(np.array(wine.feature_names)[keepers_idx])
```

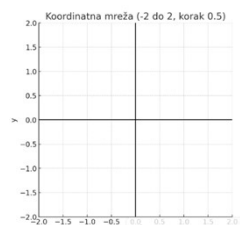
Nelinearne veze. Regresija – **mutual_info_regression**

```
ftrsel = ftr_sel.SelectPercentile(ftr_sel.mutual_info_regression,
                                  percentile=25)
ftrsel.fit_transform(diabetes_train_fts,
                    diabetes_train_tgt)

print(np.array(diabetes.feature_names)[ftrsel.get_support()]])
```

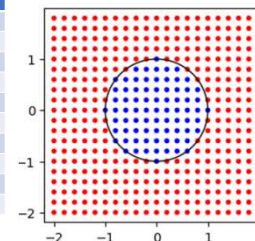
13.2. SVM – kernel metoda

X	Y	klasa
0	0	P
0,5	0,5	P
1	1	C
-1	-1	C
-0,5	-0,5	P
-0,5	0,5	P
0,5	-0,5	P
2	2	C
-2	-2	C
2	-2	C

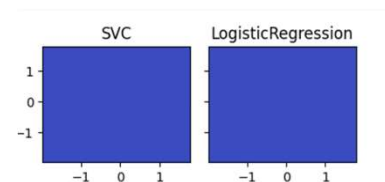


13.2. SVM – kernel metoda

X	Y	klasa
0	0	P
0,5	0,5	P
1	1	C
-1	-1	C
-0,5	-0,5	P
-0,5	0,5	P
0,5	-0,5	P
2	2	C
-2	-2	C
2	-2	C



```
shootout_linear = [svm.SVC(kernel='linear'),
                    linear_model.LogisticRegression()]
fig, axes = plt.subplots(1,2,figsize=(4,2), sharey=True)
for mod, ax in zip(shootout_linear, axes):
    plot_boundary(ax, data, tgt, mod, [0,1])
    ax.set_title(get_model_name(mod))
plt.tight_layout()
```



```

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import f1_score

X=data
y = np.asarray(tgt)
for model in shootout_linear:
    pipe = Pipeline([
        ('scaler', StandardScaler()),
        ('clf', model)
    ])
    pipe.fit(X, y)
    y_hat = pipe.predict(X)
    f1_macro = f1_score(y, y_hat, average='macro')
    print(f"{model.__class__.__name__}: F1 (macro) = {f1_macro:.4f}")

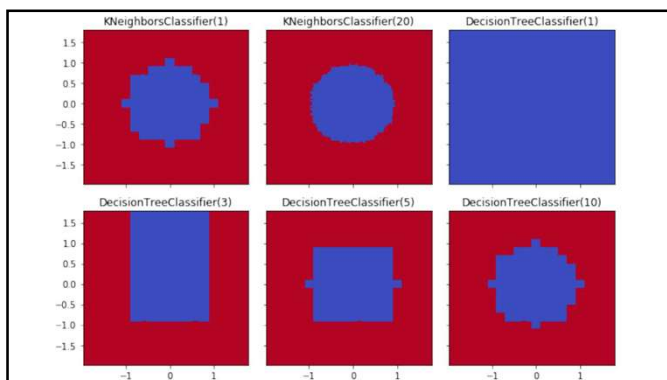
```

```

knc_p, dtc_p = [1,20], [1,3,5,10]
KNC = neighbors.KNeighborsClassifier
DTC = tree.DecisionTreeClassifier
shootout_nonlin = ([KNC(n_neighbors=p), p] for p in knc_p) +
[(DTC(max_depth=p), p) for p in dtc_p ]

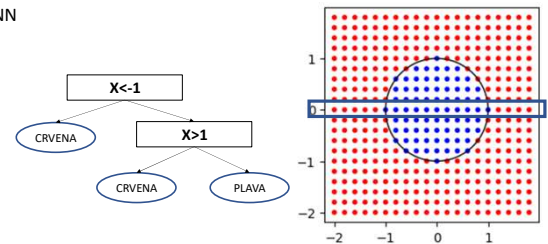
fig, axes = plt.subplots(2,3,figsize=(9, 6),
    sharex=True, sharey=True)
for (mod, param), ax in zip(shootout_nonlin, axes.flat):
    plot_boundary(ax, data, tgt, mod, [0,1])
    ax.set_title(get_model_name(mod) + "({})".format(param))
plt.tight_layout()

```



Kako k-NN i DT podržavaju nelinearnost

- K-NN
- DT



Dodavanje značajki – to znamo



```

new_data = np.concatenate([data, data**2], axis=1)
print("Novi oblik podataka:", new_data.shape)
print("Prvi red:", new_data[0])

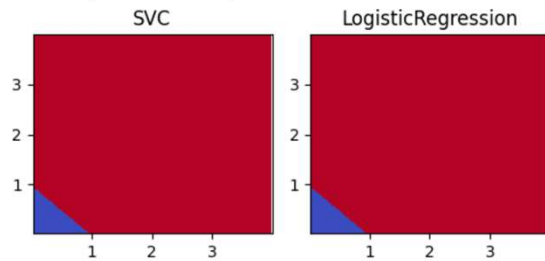
```

```

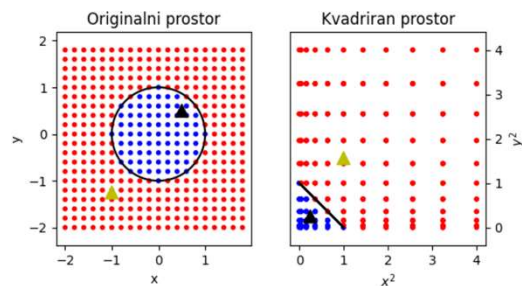
fig, axes = plt.subplots(1,2,figsize=(5,2.5))
for mod, ax in zip(shootout_linear, axes):
    plot_boundary(ax, new_data, tgt, mod, [2,3])
    ax.set_title(get_model_name(mod))
plt.tight_layout()

```

Novi oblik podataka: (400, 4)
Prvi red: [-2. -2. 4. 4.]



```
X=new_data
y = np.asarray(tgt)
for model in shootout_linear:
    pipe = Pipeline([
        ('scaler', StandardScaler()),
        ('clf', model)
    ])
    pipe.fit(X, y)
    y_hat = pipe.predict(X)
    f1_macro = f1_score(y, y_hat, average='macro')
    print(f"{model.__class__.__name__}: F1 (macro) = {f1_macro:.4f}")
```



`metrics.pairwise.polynomial_kernel`

```
k_data = metrics.pairwise.polynomial_kernel(data, data,
                                             degree=2) # squares

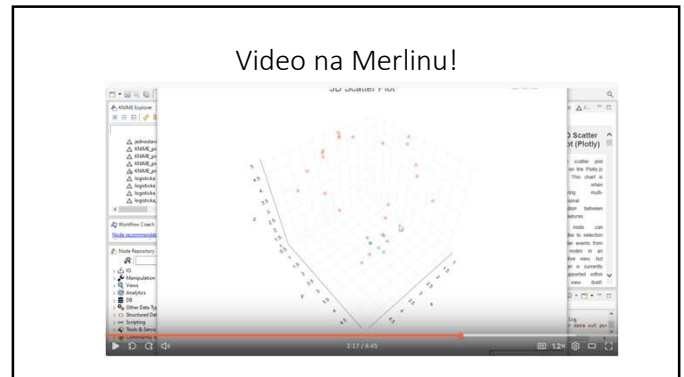
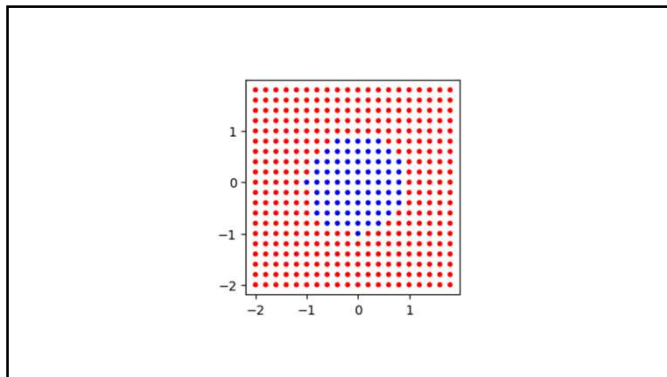
print('first example: ', data[0])
print('first example in kernelized features:', k_data[0])
print('example in original features:', data[0].shape)
print('example in kernelized features:', k_data[0].shape)
print('# examples in both:', len(data), len(k_data))
```

```
X=k_data
y = np.asarray(tgt)
for model in shootout_linear:
    pipe = Pipeline([
        ('scaler', StandardScaler()),
        ('clf', model)
    ])
    pipe.fit(X, y)
    y_hat = pipe.predict(X)
    f1_macro = f1_score(y, y_hat, average='macro')
    print(f"{model.__class__.__name__}: F1 (macro) = {f1_macro:.4f}")
```

```
fig, ax = plt.subplots(figsize=(4,3))

# treniramo model sa k_data umjesto originalnih
preds = (linear_model.LogisticRegression()
         .fit(k_data, tgt)
         .predict(k_data))

ax.scatter(xs, ys, c=np.where(preds, 'r', 'b'), marker='.')
ax.set_aspect('equal')
```



Kako radi kernalizacija ?

1) Podaci (3x2)

Neka su tri uzorka:

- $x_1 = (1, 0)$
- $x_2 = (0, 1)$
- $x_3 = (1, 1)$

Zapišimo ih kao matricu $X \in \mathbb{R}^{3 \times 2}$:

2) Odaberimo kernel (polinomni, stupanj 2)

Uzet ćemo polinomni kernel

$$K(x, y) = (x^T y + 1)^2.$$

Poanta kernalizacije: **nećemo eksplicitno dodavati nove (kvadratne) značajke**; umjesto toga računamo Gramovu (kernel) matricu K čiji elementi su $K_{ij} = K(x_i, x_j)$.

3) Račun K bez proširivanja značajki

Prvo skalarni produkti:

- $x_1 \cdot x_1 = 1, x_1 \cdot x_2 = 0, x_1 \cdot x_3 = 1$
- $x_2 \cdot x_2 = 1, x_2 \cdot x_3 = 1$
- $x_3 \cdot x_3 = 2$

Sada kernel $K_{ij} = (x_i \cdot x_j + 1)^2$:

- $K_{11} = (1 + 1)^2 = 4, K_{12} = (0 + 1)^2 = 1, K_{13} = (1 + 1)^2 = 4$
- $K_{22} = (1 + 1)^2 = 4, K_{23} = (1 + 1)^2 = 4$
- $K_{33} = (2 + 1)^2 = 9$

Dakle

$$K = \begin{bmatrix} 4 & 1 & 4 \\ 1 & 4 & 4 \\ 4 & 4 & 9 \end{bmatrix}.$$

Ova matrica K je sve što trebaju **kernel metode** (npr. SVM s polinomnim kernelom, kernel PCA, kernel ridge...). One rade "kao da" smo podigli podatke u višu dimenziju s kvadratnim i križnim članovima — ali bez da smo te stupce ikad eksplicitno dodali.

$$\begin{aligned} x_1 &= (1, 0) \\ x_2 &= (0, 1) \\ x_3 &= (1, 1) \end{aligned}$$

SVM – metoda potpornih vektora (uz korištenje kernela!)

- Kerneli: linear, poly, rbf,...

- Linear**: visoka dimenzionalnost, ako se očekuje linearno razdvajanje
- RBF**: kada je potrebna univerzalna nelinearnost ali obavezno koristiti grid-search za C i gamma (npr. gamma="scale" kao start).
- Poly**: postojanje polinomnih odnosa niskog reda (manji degree (2–3))

Hiperparametri ?

- C (sva tri): penalizira pogreške; veći C → manje regularizacije (veći rizik overfita).
- gamma (poly, rbf): "domet" sličnosti; veći gamma → granica detaljnija/"oštrija".
- degree (poly): stupanj polinoma; veći → složeniji model.

```

from sklearn import svm
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score

```

```
X, y = load_digits(return_X_y=True)
```

```
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

```

sv_classifiers = {
    "LinearSVC" : svm.LinearSVC(),
    "SVC(linear)" : svm.SVC(kernel='linear'),
    "SVC(poly=1)" : svm.SVC(kernel='poly', degree=1),
    "SVC(poly=2)" : svm.SVC(kernel='poly', degree=2),
    "SVC(poly=3)" : svm.SVC(kernel='poly', degree=3),
    "SVC(rbf,.5)" : svm.SVC(kernel='rbf', gamma=0.5),
    "SVC(rbf,1.0)" : svm.SVC(kernel='rbf', gamma=1.0),
    "SVC(rbf,2.0)" : svm.SVC(kernel='rbf', gamma=2.0)}

```

```

for name, clf in sv_classifiers.items():
    pipe = Pipeline([("scaler", StandardScaler()), ("clf", clf)])
    pipe.fit(X_tr, y_tr)
    acc = accuracy_score(y_te, pipe.predict(X_te))
    print(f"{name:12s} | accuracy = {acc:.4f}")

```

LinearSVC	accuracy = 0.9556
SVC(linear)	accuracy = 0.9750
SVC(poly=1)	accuracy = 0.9861
SVC(poly=2)	accuracy = 0.9778
SVC(poly=3)	accuracy = 0.9722
SVC(rbf,.5)	accuracy = 0.3861
SVC(rbf,1.0)	accuracy = 0.1361
SVC(rbf,2.0)	accuracy = 0.1167



```

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

```

```
X, y = load_digits(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

```

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", SVC(kernel="rbf"))])

```

```

param_grid = {
    "clf__C": [0.1, 1, 10, 100],
    "clf__gamma": ["scale", "auto", 0.01, 0.1, 1, 10]}

```

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```

grid = GridSearchCV(
    estimator=pipe,
    param_grid=param_grid,
    scoring="accuracy",
    cv=cv,
    n_jobs=-1)

```

```

grid.fit(X_tr, y_tr)
y_pred = grid.best_estimator_.predict(X_te)
print(f"{accuracy_score(y_te, y_pred):.4f}")
print(grid.best_params_)

```



```

0.9750
{'clf__C': 1, 'clf__gamma': 'auto'}

```

```

from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, GridSearchCV, KFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVR
from sklearn.metrics import r2_score

```

```

X, y = load_diabetes(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)

```

```

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("svr", SVR())])

```



```
# Grid s tri kernela, svaki s po 2 hiperparametra
param_grid = [
    # linear: C i epsilon
    {"svr__kernel": ["linear"],
     "svr__C": [0.1, 10],
     "svr__epsilon": [0.1, 0.2]},
    # poly: C i degree
    {"svr__kernel": ["poly"],
     "svr__C": [1, 10],
     "svr__degree": [2, 3]},
    # rbf: C i gamma
    {"svr__kernel": ["rbf"],
     "svr__C": [1, 100],
     "svr__gamma": ["scale", 0.1]]
```

```
cv = KFold(n_splits=5, shuffle=True, random_state=42)

grid = GridSearchCV(
    estimator=pipe,
    param_grid=param_grid,
    scoring="r2",
    cv=cv,
    n_jobs=-1,
    refit=True,)

grid.fit(X_tr, y_tr)
y_pred = grid.best_estimator_.predict(X_te)
print(f"r2_score(y_te, y_pred): .4f")
```

PREPORUKE za SVM

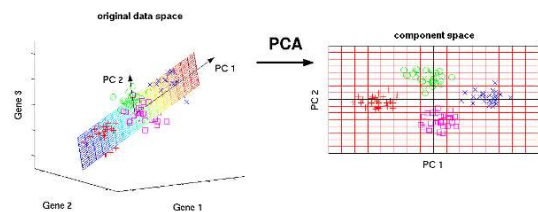
- Skalirajte podatke
- Koristite RBF kernel
- Odaberite parametre jezgre pomoću unakrsne provjere



Preporuke Andrew Ng:

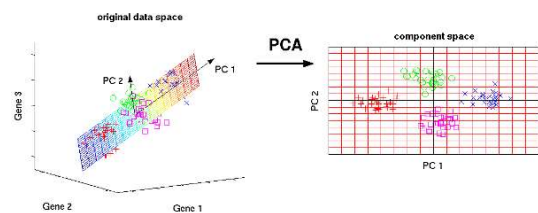
- Koristite linearnu jezgru kada je broj značajki veći od broja zapažanja
- Koristite RBF kernel kada je broj opažanja veći od broja značajki
- Ako postoji puno opažanja, recimo više od 50k, razmislite o korištenju linearne jezgre za ubrzanje

13.3. PCA metoda - (Principal Component Analysis)



Za što se koristi ?

- Smanjenje dimenzionalnosti (brži modeli, manje overfittinga).
- Vizualizacija višedimenzionalnih podataka u 2D/3D.
- Uklanjanje šuma (zadržavanje dominantne komponente).
- Kompresija podataka (manje značajki uz minimalan gubitak informacije).
- Feature engineering (PC komponente kao nove značajke).



```

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

X, y = load_iris(return_X_y=True)
target_names = ["setosa", "versicolor", "virginica"]

X_std = StandardScaler().fit_transform(X)

pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_std)

print("Postotak varijance po komponenti:", pca.explained_variance_ratio_)
print("Kumulativno objašnjena varijanca:", pca.explained_variance_ratio_.sum())

plt.figure(figsize=(6,5))
for i, name in enumerate(target_names):
    plt.scatter(X_pca[y==i, 0], X_pca[y==i, 1], label=name, s=40)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("Iris - PCA (2D)")
plt.legend()
plt.tight_layout()
plt.show()

```

